ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

Knowledge reasoning with knowledge base

Yiru Zhang

Web mining & semantics

Département d'Informatique

Ecole Supérieure d'Ingénieurs Léonard de Vinci

DE VINCI
RESEARCH CENTER

Roadmap

1. Overview
2. Symbolic / Logic-based reasoning
3. Embedding reasoning
4. Hybrid reasoning (brief)

What is reasoning

- A **reasoner** infers knowledge that is **not explicitly stated** in the ontology.
  - Think of OWL as a structured knowledge base.
    A reasoner is the “logic engine” that derives hidden consequences.
- Two kinds of reasoning
  - Terminological Reasoning (TBox reasoning): Reasoning about **classes and hierarchy**
  - Instance Reasoning (ABox reasoning): Reasoning about **individuals**

ESILV

Terminological Reasoning (TBox reasoning)

- Reasoning about **classes and hierarchy**
- Tasks:
  - Consistency checking: Detect contradictory definitions
    - **Defining** Man ⊑ Woman and Man ⊑ ¬Woman
  - Classification: Compute direct super/sub-classes
    - **Discover** PhDStudent ⊑ Student
  - Taxonomy construction: Build complete class hierarchy
    - Infer implicit subclass relations
- Example:

PhDStudent ⊑ Student
Student ⊑ Person

The reasoner infers: → PhDStudent ⊑ Person

ESILV

Instance Reasoning (ABox reasoning)

- Reasoning about **individuals**
- Tasks:
  - Instance checking: Does individual belong to class?
    - Is john a Student?
  - Individual retrieval: Find all instances of a class
    - Retrieve all HappyStudent
- Example:
  - If Student(john) and Student $\sqsubseteq$ Person
  - The reasoner infers: Person(john)

ESILV

Rule-based inference

Motivation: Why Do We Need Rules?

- OWL is powerful — but not sufficient for all modeling needs.
- Limitations of OWL
  - Cannot easily express certain conditional logic
  - Limited expressiveness for composite relations
  - Designed for decidability (so some expressive power is sacrificed)
- Example of the limitation:
  - We want to express: “If a person has a parent who has a brother, then that brother is the person’s uncle.”
  - This is difficult in pure OWL. But easy with rules:

$$\text{hasParent}(x,y) \land \text{hasBrother}(y,z) \rightarrow \text{hasUncle}(x,z)$$

ESILV

We Need Both Structure and Rules

- **Ontology Layer** (OWL-DL)
  - Based on Description Logics
  - Structured conceptual model
  - Reusable and interoperable
  - Good for class hierarchies
- **Rule Layer**
  - Based on Logic Programming
  - Express procedural knowledge
  - More expressive for certain patterns
  - Efficient reasoning engines exist

Rules (High Expressiveness)
↑
OWL Ontology
↑
RDF
↑
XML
↑
URI / Unicode

Interpretation:
•Ontology = structured domain model
•Rules = domain logic

ESILV

Write a rule

- Rules are written as **Horn clauses**.
  - General form: $A1 \wedge A2 \wedge ... \wedge An \rightarrow H$
    - $\rightarrow$ Left side = Body (Antecedent)
    - $\rightarrow$ Right side = Head (Consequent)
  - Example: $Student(x) \wedge happilyStudyingIn(x,y) \rightarrow HappyStudent(x)$
    - If x is a student and studies happily, then x is a happy student.
  - Constraints:
    - Variables allowed (with x,y)
    - Constants allowed (with $\wedge$)
    - Function symbols allowed
    - No classical negation

ESILV

Where are the quantifiers? (universal? Existential?)

- Quantifiers are implicit. **Variables in body** implies **universally** quantified, while **variables only in head** implies **existentially** quantified
  - isParent(X) → hasChild(X,Y) is equivalent to:
  - ∀X (isParent(X) → ∃Y hasChild(X,Y)). Meaning “Every parent has at least one child.”

ESILV

Semantic Web Rule Language (SWRL): Rules + Ongologies integration

- SWRL is a rule language designed for OWL-DL (Description Logic). It allows Horn-like rules combined with OWL
- Example:
  - Person(?p) ∧ hasSibling(?p,?s) ∧ Man(?s) → hasBrother(?p,?s)
  - If a person has a sibling who is a man, then that sibling is his brother.
    - ?p, ?s are variables
    - ∧ = logical AND
    - → = implication

ESILV

SWRL Built-ins

- SWRL provides built-in functions, with namespace swrlb:
- Comparison, Arithmetic, String manipulation, date, time, duration, URI and Lists
  - Older Brother? If x2 is brother of x1 and older, then x2 is older brother.

```
hasBrother(?x1,?x2)
∧ hasAge(?x1,?age1)
∧ hasAge(?x2,?age2)
∧ swrlb:greaterThan(?age2,?age1)
→ hasOlderBrother(?x1,?x2)
```

  - Decade older brother? Computes age difference

```
swrlb:subtract(10,?age2,?age1)
```

- More built-ins on: https://www.daml.org/rules/proposal/builtins.html

ESILV

SWRL on RDF in practice

- SWRL rules are typically executed by an OWL reasoner (Pellet, HermiT, etc.)
- **Owlready2** is a python module requiring Java support, supporting **Pellet/HermiT** (You can also use **Apache JENA** for reasoning. Out of the scope of this lecture)

- To see in the lab session.

ESILV

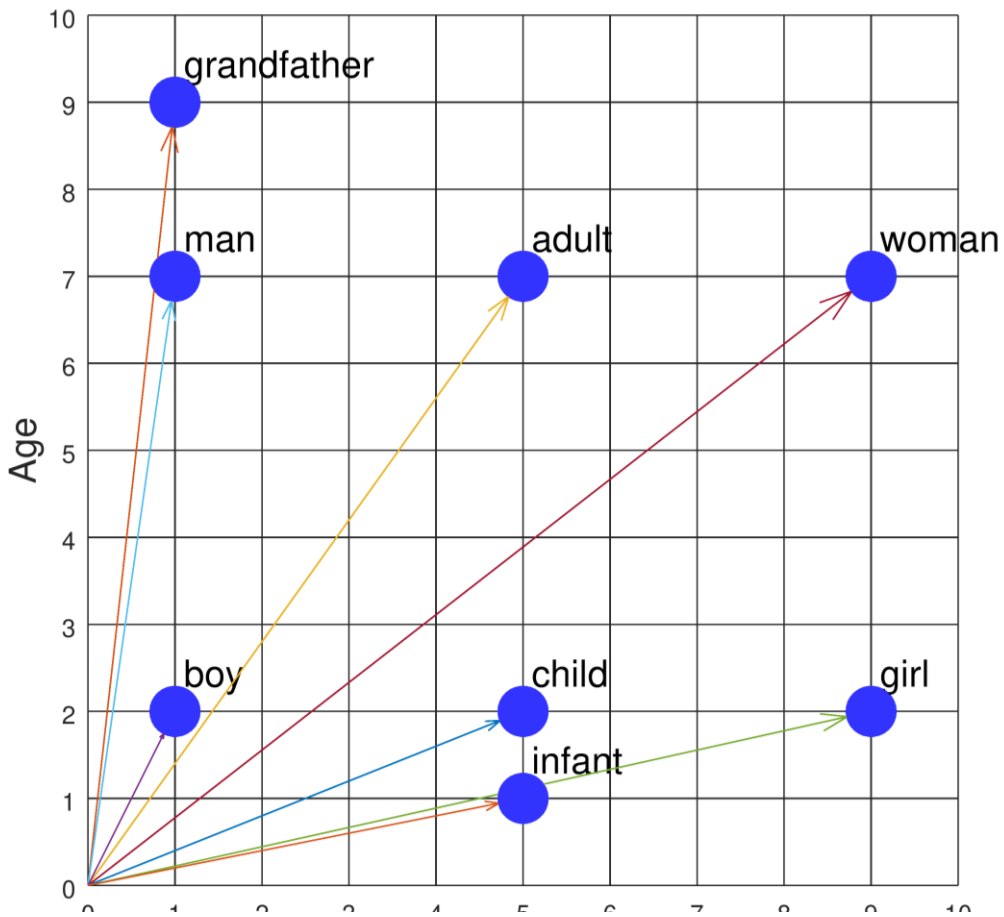
Embedding-based reasoning

- From word embedding to knowledge graph embedding
- (The embedding is a lager topic rather than a reasoning method.)

荃者所以在鱼，得鱼而忘荃
Nets are for fish; Once you get the fish, you can forget the net.
言者所以在意，得意而忘言
Words are for meaning; Once you get the meaning, you can forget the words
-庄子(Zhuangzi), Chapter 26

ESILV

Word embedding



How to represent a word?

Problem: distance between words using one-hot encodings always the same

dog	1	$[1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]$
cat	2	$[0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]$
person	3	$[0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]$

Idea: Instead of one-hot-encoding use a histogram of commonly co-occurring words.

ESILV

Distributional Semantics

Distributional hypothesis: words that occur in similar contexts tend to have similar meanings

Dogs are man's best friend.

I saw a dog on a leash walking in the park.

His dog is his best companion.

He walks his dog in the late afternoon

…

	friend	leash	park	walking	walks	food	legs	runs	sleeps	sits	…	
dog	[3	2	3	4	2	4	3	5	6	7	…]	

ESILV

Distributional Semantics

	food	walks	window	runs	mouse	invented	legs	sleeps	mirror	tail	...
dog	[5	5	0	5	0	0	5	5	0	2	...]
cat	[5	4	1	4	2	0	3	4	0	3	...]
person	[5	5	1	5	0	2	5	5	0	0	...]

This vocabulary can be extremely large

ESILV

Toward more Compact Representations

dog	food	walks	window	runs	mouse	invented	legs	sleeps	mirror	tail	...
cat	[5	4	1	4	2	0	3	4	0	3	...]
person	[5	5	1	5	0	2	5	5	0	0	...]
dog	[5	5	0	5	0	0	5	5	0	2	...]

This vocabulary can be extremely large

ESILV

Toward more Compact Representations

$$\text{dog} = \begin{bmatrix} 5 \\ 5 \\ 0 \\ 5 \\ 0 \\ 0 \\ 5 \\ 5 \\ 0 \\ 2 \\ \dots \end{bmatrix} = w1 \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ \dots \end{bmatrix} + w2 \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ \dots \end{bmatrix} + w3 \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ \dots \end{bmatrix} + \dots$$

legs, running, walking

tail, fur, ears

mirror, window, door

ESILV

Toward more Compact Representations

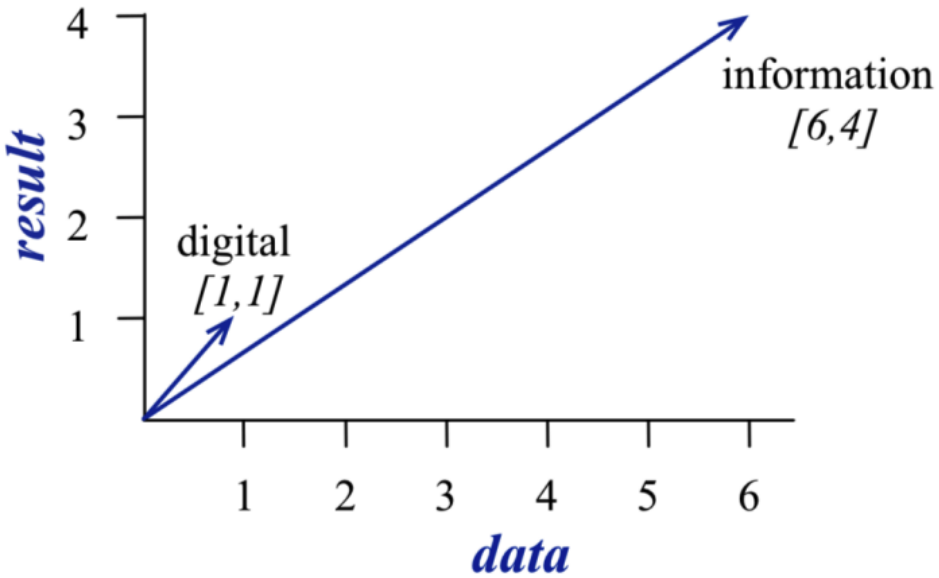
dog = [w1 w2 w3]

The basis vectors can be found using Principal Component Analysis (PCA)

ESILV

Words as vectors

- We'll build a new model of meaning focusing on similarity
  - Each word is a vector
  - Similar words are "nearby in space"
- A first solution: we can just use context vectors to represent the meaning of words!



$$cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$$

$$cos(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^{V} u_i v_i}{\sqrt{\sum_{i=1}^{V} u_i^2}\sqrt{\sum_{i=1}^{V} v_i^2}}$$

Words as vectors

- Problem: not all counts are equal, words can randomly co-occur
  - Solution: re-weight by how likely it is for the two words to co-occur by simple chance
  - PPMI = Positive Pointwise Mutual Information

$$PPMI(w,c) = \max(log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

PPMI compares the probability of two events occurring together to what this probability would be if the events were independent.

	computer	data	result	pie	sugar
cherry	2	8	9	442	25
strawberry	0	0	1	60	19
digital	1670	1683	85	5	4
information	3325	3982	378	5	13

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

ESILV

Sparse vs. Dense vectors

- Still, the vectors we get from word-word occurrence matrix are sparse (most are 0's) & long (vocabulary size)
- Alternative: we want to represent words as short (50-300 dimensional) & dense (real-valued) vectors

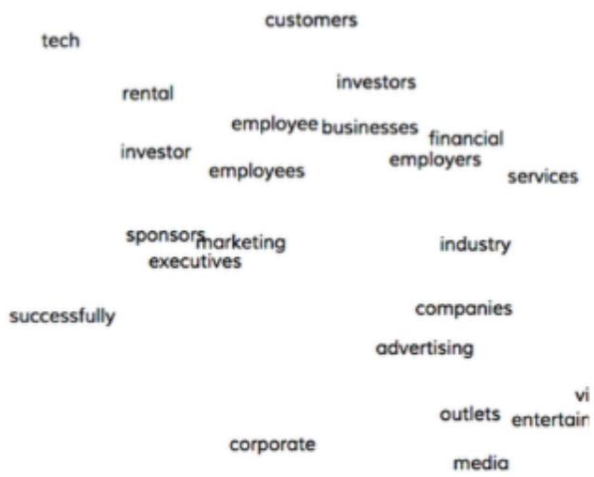
	computer	data	result	pie	sugar
cherry	2	8	9	442	25
strawberry	0	0	1	60	19
digital	1670	1683	85	5	4
information	3325	3982	378	5	13

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

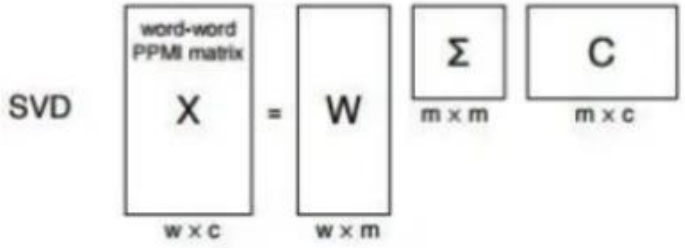
$$\text{employees} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 10.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$

Dense vectors

$$\text{employees} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 10.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$



- Short vectors are easier to use as features in ML systems
- Dense vectors may generalize better than storing explicit counts
- They do better at capturing synonymy
  - w_1 co-occurs with “car”, w_2 co-occurs with “automobile”
- Different methods for obtaining dense vectors:
  - Singular value decomposition (SVD)
  - Word2vec (and modified versions): “learn” the vectors



What is `word2vec`?

- `word2vec` is **not** a single algorithm
- It is a **software package** for representing words as vectors, containing:
  - Two distinct models
    - CBoW
    - **Skip-Gram** **(SG)**
  - Various training methods
    - **Negative Sampling** **(NS)**
    - Hierarchical Softmax
  - A rich preprocessing pipeline
    - Dynamic Context Windows
    - Subsampling
    - Deleting Rare Words

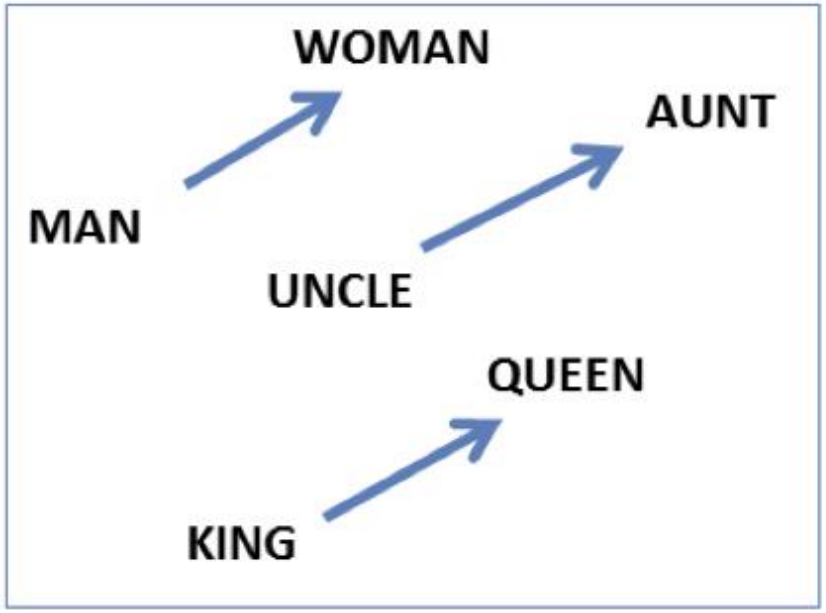
Slide by Omer Levy

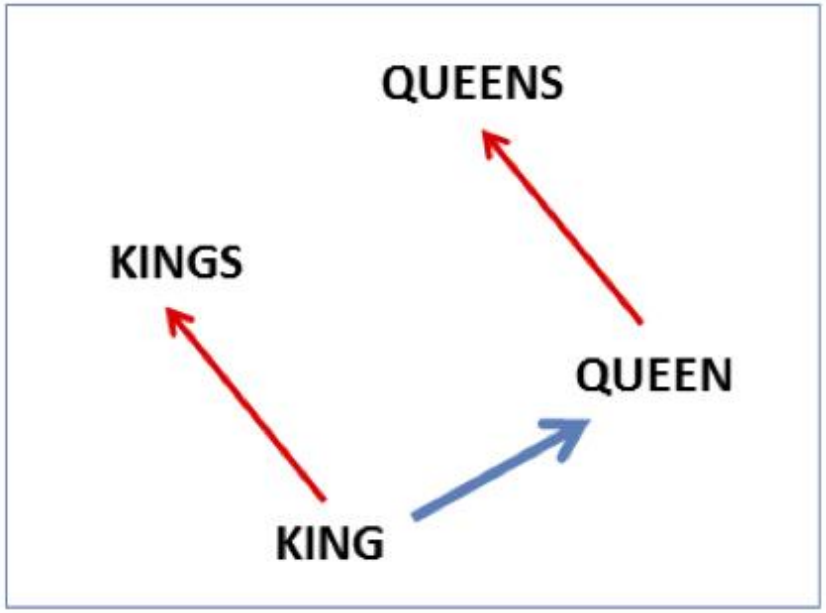
ESILV

Embedding vectors capture relational meaning!

vector(*'king'*) - vector(*'man'*) + vector(*'woman'*) ≈ vector('queen')

vector(*'Paris'*) - vector(*'France'*) + vector(*'Italy'*) ≈ vector('Rome')





Slide by Dan Jurafsky

Word2Vec: Main idea

- Use a classifier to predict which words appear in the context of a target word (or vice versa)
  - This classifier induces a dense vector representation of words (embedding)
  - Words that appear in similar contexts (that have high distributional similarity) will have very similar vector representations.
- These models can be trained on large amounts of raw text

ESILV

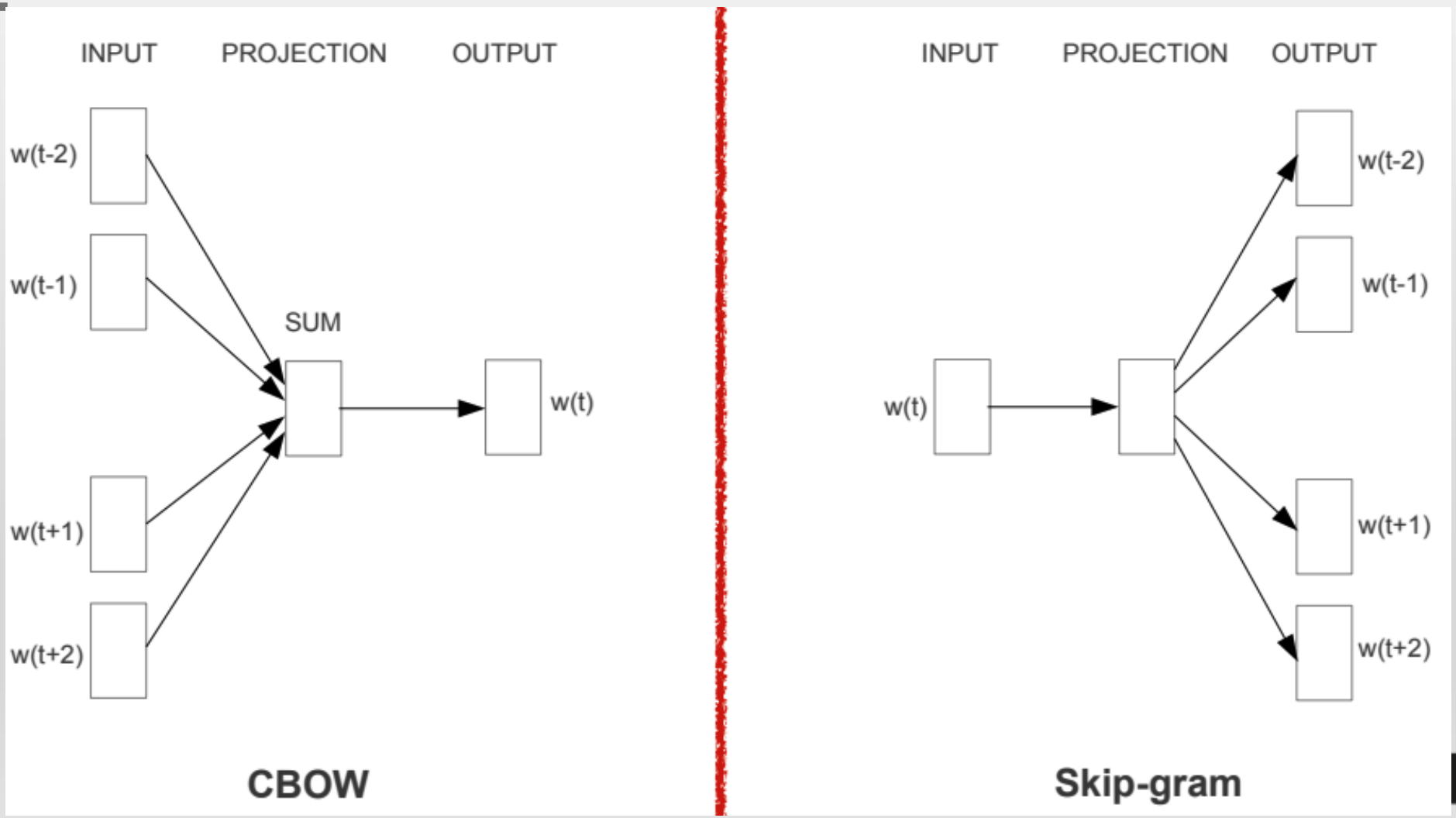
Word2Vec: Problem setting

- Input: a large *Text corpora*, *V*, *d*
  - *V*: a pre-defined vocabulary
  - *d*: dimension of word vectors (e.g. 300)
  - *Text corpora*:
    - Wikipedia + Gigaword 5: 6B
    - Twitter: 27B
    - Common Crawl: 840B
- Output: $f: V \rightarrow \mathbb{R}^d$

$$v_{\text{cat}} = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ 0.276 \end{pmatrix} \quad v_{\text{dog}} = \begin{pmatrix} -0.124 \\ 0.430 \\ -0.200 \\ 0.329 \end{pmatrix}$$

$$v_{\text{the}} = \begin{pmatrix} 0.234 \\ 0.266 \\ 0.239 \\ -0.199 \end{pmatrix} \quad v_{\text{language}} = \begin{pmatrix} 0.290 \\ -0.441 \\ 0.762 \\ 0.982 \end{pmatrix}$$

Word2Vec architectures



Continuous Bag of Words (CBoW)

- Predict target from text ... lemon, a **tablespoon of apricot jam a** pinch ...

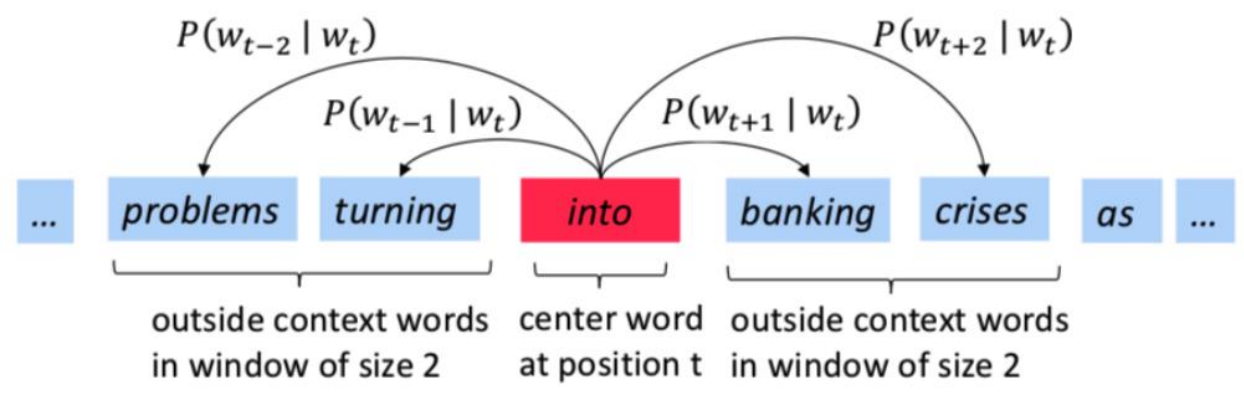
  c1 c2 t c3 c4

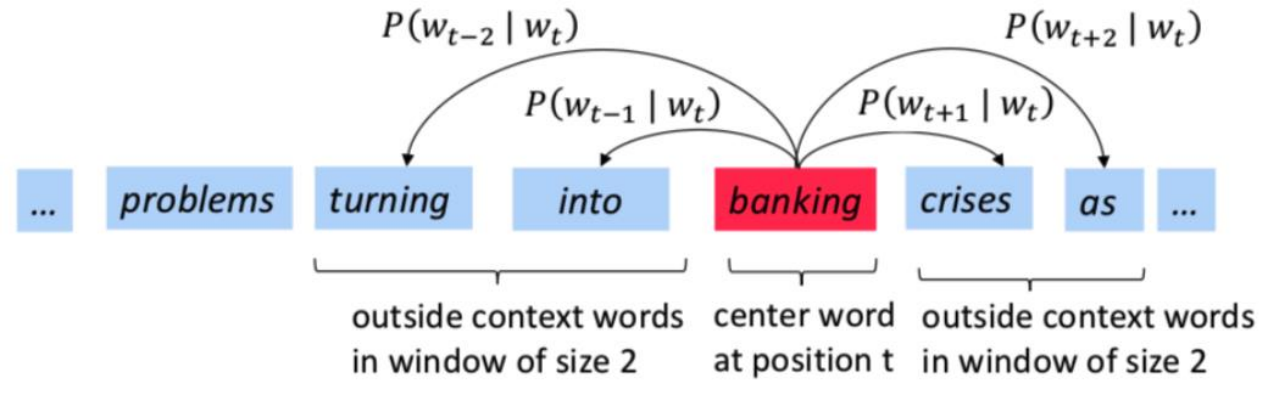
- Given the surrounding context words (tablespoon, of, jam, a), predict the target word (apricot)
- **Input**: each context word is a one-hot vector
- **Projection layer**: map each one-hot vector down to a dense D-dimensional vector, and average these vectors
- **Output**: predict the target word with softmax

ESILV

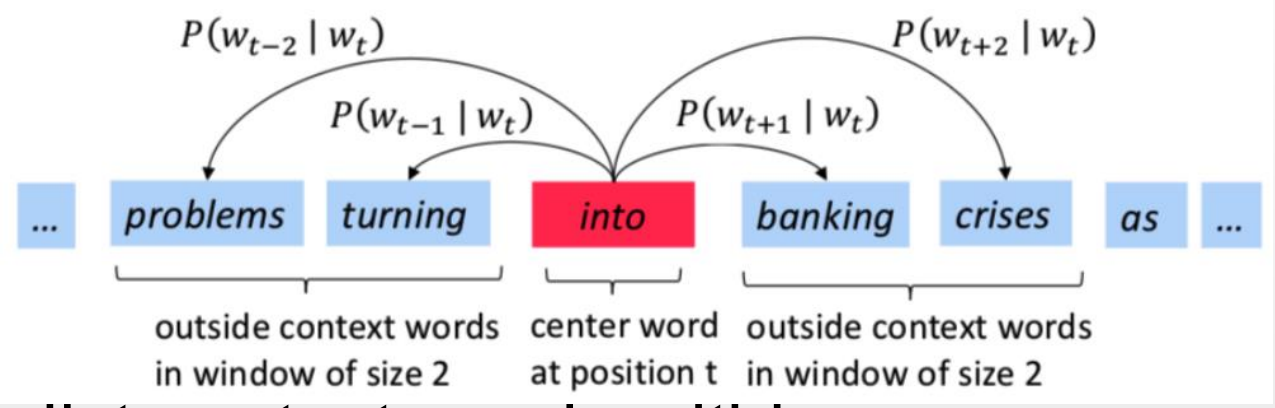
Skip-gram: idea

- We want to use words to predict their context words
- Context: a fixed window of size *2m*





Word2Vec Objective



- For each position *t=1,2,…T*, predict context words within context size *m*, given center word w_j:

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m, j \ne 0} P(w_{t+j} | w_t; \theta)$$

θ :all the parameters to be optimized

- The objective function *J(θ)* is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{-m \le j \le m, j \ne 0} \log P(w_{t+j} | w_t; \theta)$$

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-m\le j\le m, j\neq 0} \log P(w_{t+j}|w_t;\theta)$$

Word2Vec Objective

- How to define $P(w_{t+j}|w_t;\theta)$?
- We have two sets of vectors for each word in the vocabulary
  - $\mathbf{u}_i \in \mathbb{R}^d$: embedding for target word *i*
  - $\mathbf{v}_{i'} \in \mathbb{R}^d$: embedding for context word *i'*
- Use inner product $\mathbf{u}_i^T\mathbf{v}_{i'}$ to measure how likely word *i* appears with context word *i'*, the larger the better

$$P(w_{t+j}|w_t) = \frac{\exp({\mathbf{u}_{w_t}}^{\mathrm{T}}\mathbf{v}_{w_{t+j}})}{\sum_{k\in V}\exp({\mathbf{u}_{w_t}}^{\mathrm{T}}\mathbf{v}_k)}$$

"softmax"

$\theta = \{\{\mathbf{u}_k\},\{\mathbf{v}_k\}\}$ are all the parameters in this model!

ESILV

Training process

- Calculating all the gradients together!

$$\theta = \{\{\mathbf{u}_k\}, \{\mathbf{v}_k\}\}$$

$$J(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{-m\leq j\leq m, j\neq 0} \log P(w_{t+j}|w_t;\theta)$$

$$\nabla_\theta J(\theta) = ?$$

- We can apply SGD! (or other optimization algorithms):

$$\theta^{(t+1)} = \theta^t - \eta\nabla_\theta J(\theta)$$

Skip-Gram Training data

- Training sentence:

... lemon, a tablespoon of apricot jam a pinch ...
c1 c2 t c3 c4

- Training data: input/output pairs centering on apricot
  - Assume a +/- 2 word window (in reality: use +/- 10 words)
  - **Positive examples**: (apricot, tablespoon), (apricot, of), (apricot, jam), (apricot, a)
  - For each positive example, sample k **negative examples**, using noise words (according to [adjusted] unigram probability) (apricot, aardvark), (apricot, puddle)…

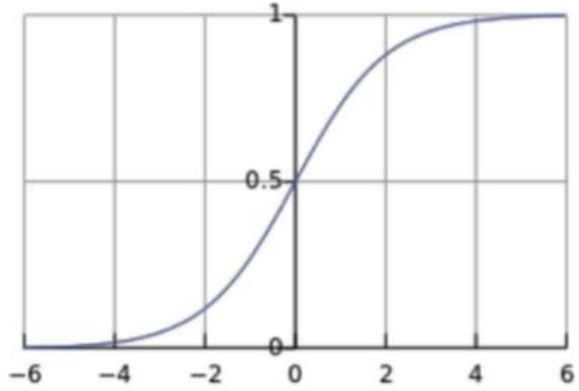
Skip-Grams with Negative Sampling (SGNS)

- Idea: recast problem as binary classification!
  - Target word is positive example
  - All words not in context are negative

positive examples +		negative examples -			
t	c	t	c	t	c
apricot	tablespoon	apricot	aardvark	apricot	seven
apricot	of	apricot	my	apricot	forever
apricot	jam	apricot	where	apricot	dear
apricot	a	apricot	coaxial	apricot	if

$$P(D=1 \mid t,c) = \sigma(\mathbf{u}_t \cdot \mathbf{v}_c)$$

$$\sigma(x) = \frac{1}{1+\exp(-x)}$$

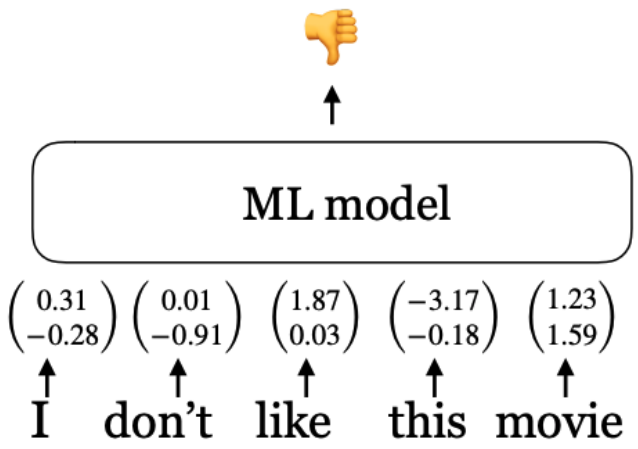


- To compute loss, pick K random words as negative examples:

$$J(\theta) = -P(D=1 \mid t,c) - \frac{1}{K}\sum_{i=1}^{K} P(D=0 \mid t_i, c)$$

Evaluating word embeddings

- Extrinsic evaluation
  - Let's plug these word embeddings into a real NLP system and see whether this improves performance
  - Could take a long time but still the most important evaluation metric
- **Intrinsic evaluation**
  - Evaluate on a specific/intermediate subtask
  - Fast to compute
  - Not clear if it really helps the downstream task



Intrinsic evaluation: word similarity

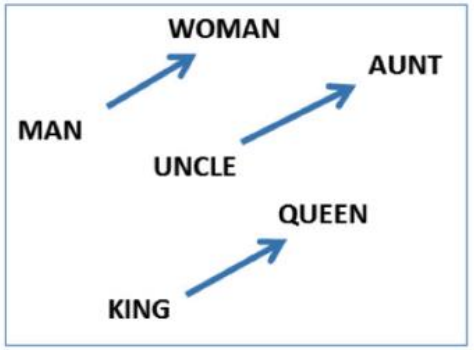
- Example dataset: wordsim-353
  - 353 pairs of words with human judgement
- Cosine similarity $\cos(\boldsymbol{u}_i, \boldsymbol{u}_j) = \frac{\boldsymbol{u}_i \cdot \boldsymbol{u}_j}{||\boldsymbol{u}_i||_2 \times ||\boldsymbol{u}_j||_2}.$
- Metric: Spearman rank correlation

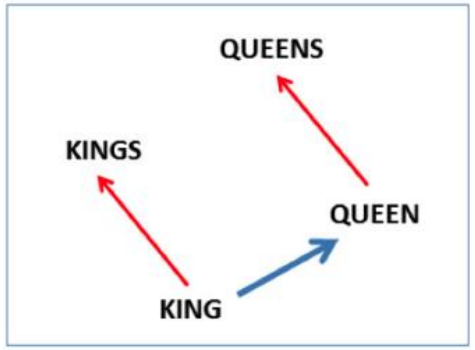
Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Model	Size	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	72.7	75.1	56.5	37.0
CBOW†	6B	57.2	65.6	68.2	57.0	32.5
SG†	6B	62.8	65.2	69.7	58.1	37.2
GloVe	6B	65.8	72.7	77.8	53.9	38.1
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	**75.9**	**83.6**	**82.9**	**59.6**	**47.8**
CBOW*	100B	68.4	79.6	75.4	59.4	45.5

Intrinsic evaluation: word analogy

- Vectors should “capture concepts”
- Analogy: embeddings capture relational meaning
  - vector(‘king’) - vector(‘man’) + vector(‘woman’) = vector(‘queen’)
  - vector(‘Paris’) - vector(‘France’) + vector(‘Italy’) = vector(‘Rome’)





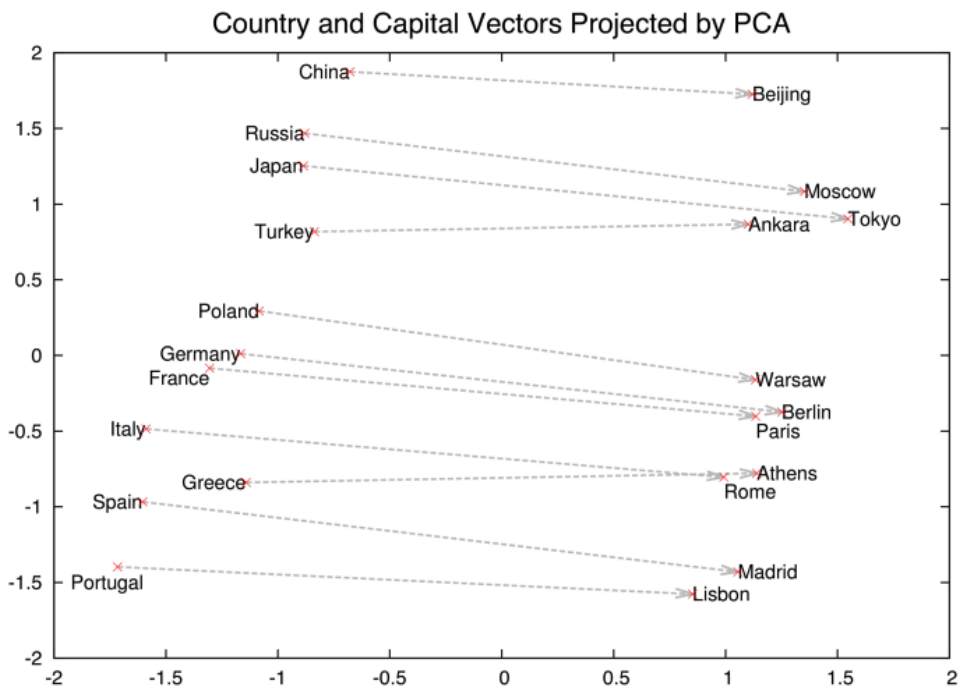


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

Some other representative work in word embedding and pretrained model

- Other works
  - Continuous Bag of words (CBOW)
  - GloVe (Global Vectors) : (Pennington et al, 2014): GloVe: Global Vectors for Word Representation
  - FastText: Sub-word Embeddings: (Bojanowski et al, 2017): Enriching Word Vectors with Subword Information
- Pretrained models:
  - word2vec: https://code.google.com/archive/p/word2vec/
  - GloVe: https://nlp.stanford.edu/projects/glove/
  - FastText: https://fasttext.cc/

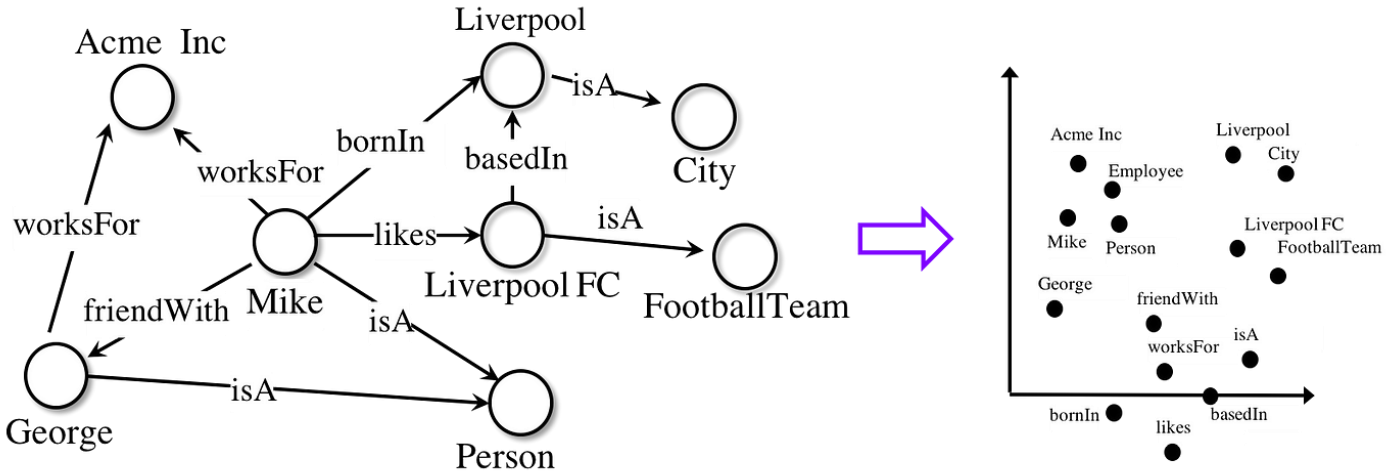
ESILV

More on embeddings

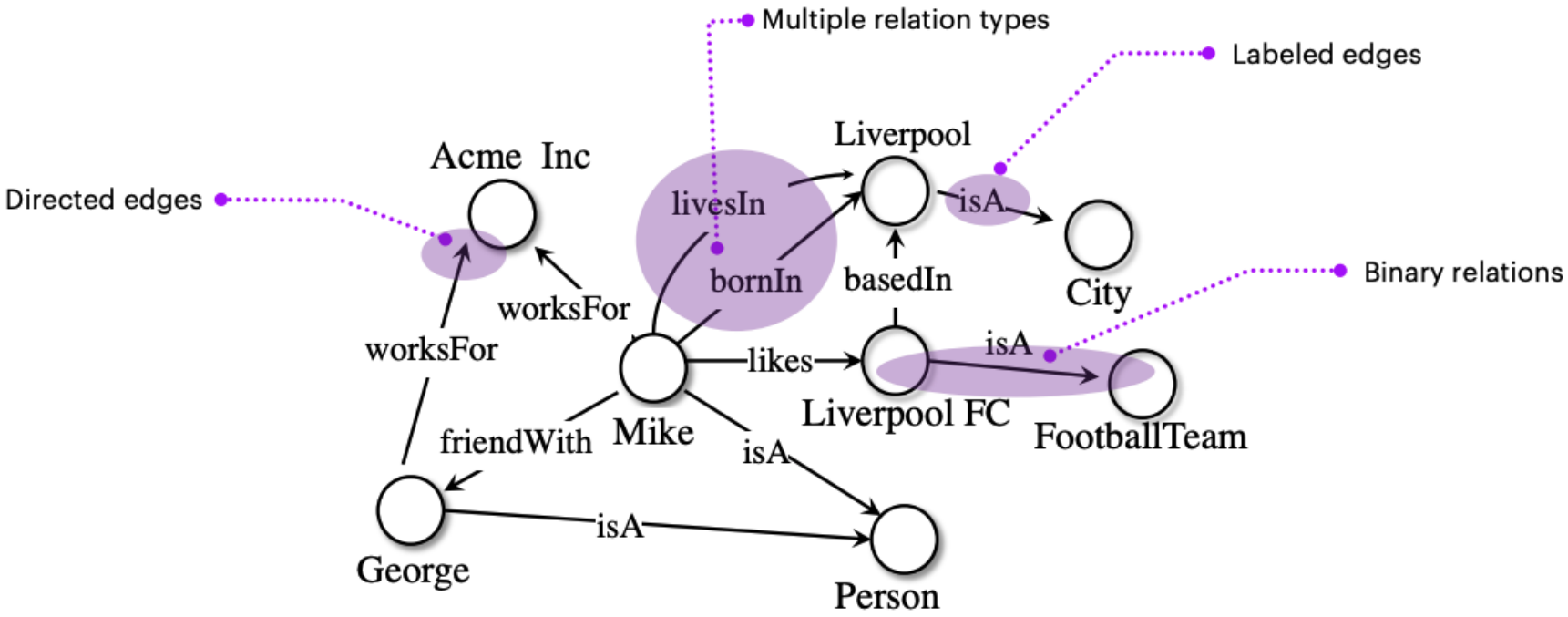
- Embeddings aren't just for words!
  - You can take any discrete input feature (with a fixed number of K outcomes, e.g. POS tags, etc.) and learn an embedding matrix for that feature.
- Where do we get the input embeddings from?
  - We can learn the embedding matrix during training.
  - Initialization matters: use random weights, but in special range (e.g. [-1/(2d), +(1/2d)] for d-dimensional embeddings), or use Xavier initialization
  - We can also use pre-trained embeddings LM-based embeddings are useful for many NLP task
  - Word embedding is the ancestor of the embedding methods used in RAG for LLM
- Embedding for knowledge graphs?

ESILV

Knowledge Graph embeddings



Recall: knowledge Graph



$$\mathcal{G} = \{(s, p, o)\} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$$

$\mathcal{E}$: set of entities of $\mathcal{G}$ $\mathcal{R}$: set of relations of $\mathcal{G}$

In-depth overview of Knowledge Graphs in [Hogan et al. 2020]

ESILV

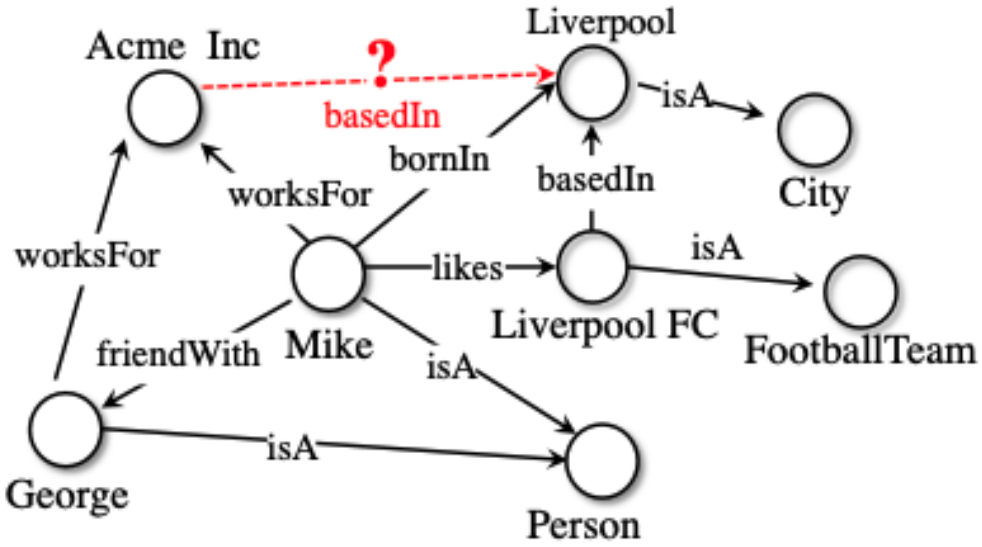
Recall: knowledge Graph

Knowledge Graph	Statements	Entities
yago select knowledge	120 M	10 M
WIKIDATA	610 M	51 M
DBpedia	1.3 B	6 M
GDELT	3.5 B	364 M

ESILV

Knowledge Graphs & The Open World Assumption

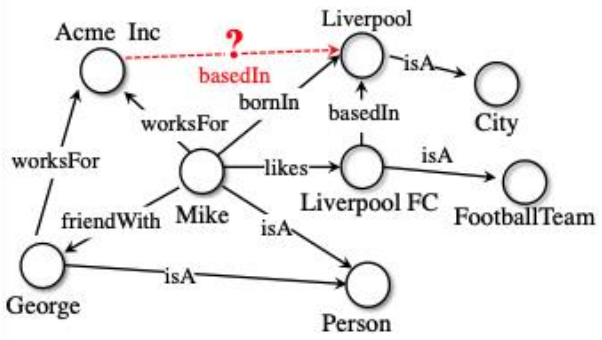
- **Closed World Assumption (CWA)**: absence of a fact means it is necessarily false.
- **Open World Assumption (OWA):** absence of a fact does not imply fact is false. We simply do not know.
  - Knowledge Graphs adopt this assumption



Machine Learning on Knowledge Graphs

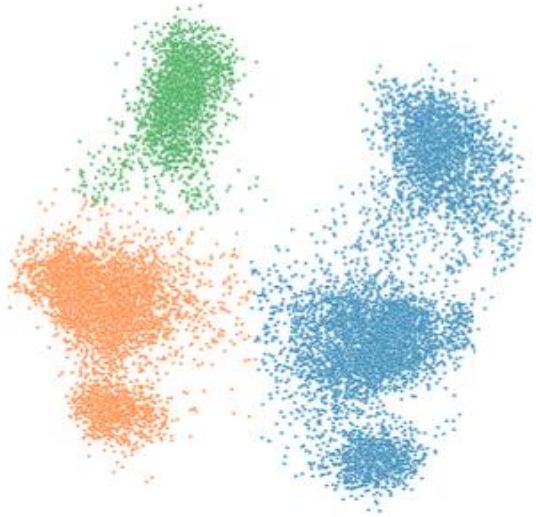
LINK PREDICTION / TRIPLE CLASSIFICATION

- Knowledge graph completion
- Content recommendation
- Question answering



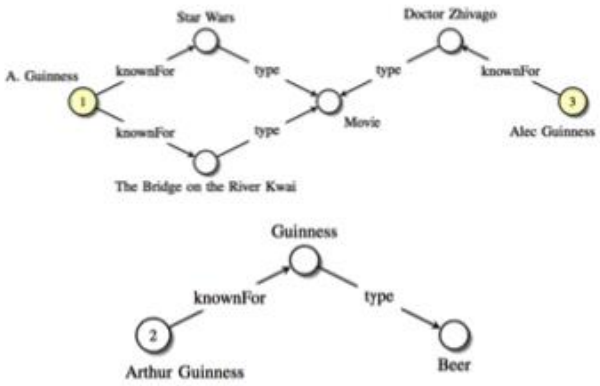
COLLECTIVE NODE CLASSIFICATION / LINK-BASED CLUSTERING

- Customer segmentation



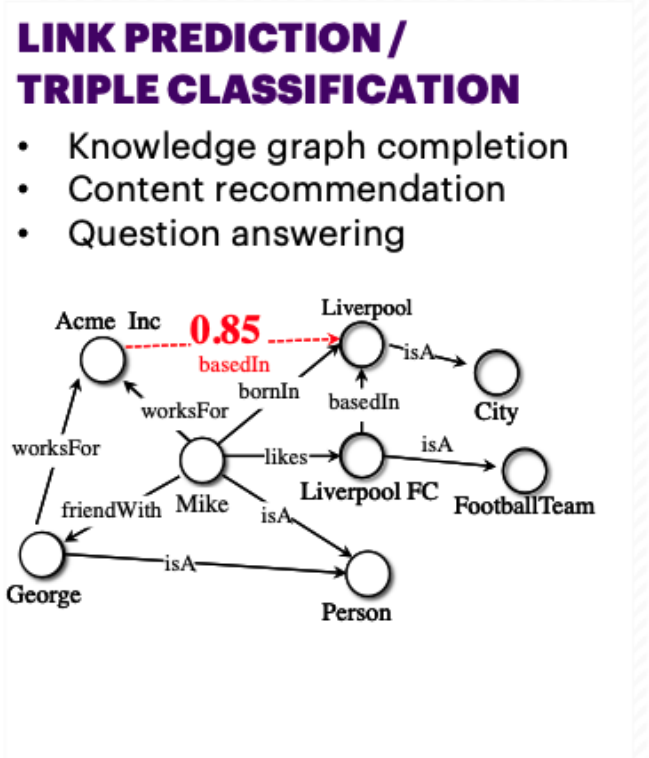
ENTITY MATCHING

- Duplicate detection
- Inventory items deduplication



Pic from [Nickel et al. 2016a]

Machine Learning on Knowledge Graphs



- Assigning a score proportional to the likelihood that an unseen triple is true. (Knowledge Graph Completion)
- **Link Prediction**
  - Learning to rank problem
  - Information retrieval metrics
  - No ground truth negatives in test set required
- **Triple Classification**
  - Binary classification task
  - Binary classification metrics
  - Test set requires positives and ground truth negatives

Traditional Statistical Relational Learning

- **Logic Programming**: predict new links from facts and extracted rules. (eg. Reasoning with OWL and SWRL)
- **Inductive Logic Programming**: predict new links from rules extracted from correlated facts.
- **Rule Mining**: e.g. AMIE+: extracts Horn clauses based on their support in the KG. [Gallaraga et al. 2015]
- Graphical models: [Getoor & Taskar 2007]
  - Conditional Random Fields (CRFs)
  - Probabilistic Relational Models
  - Relational Markov Networks
  - Relational Dependency Networks

Limitations

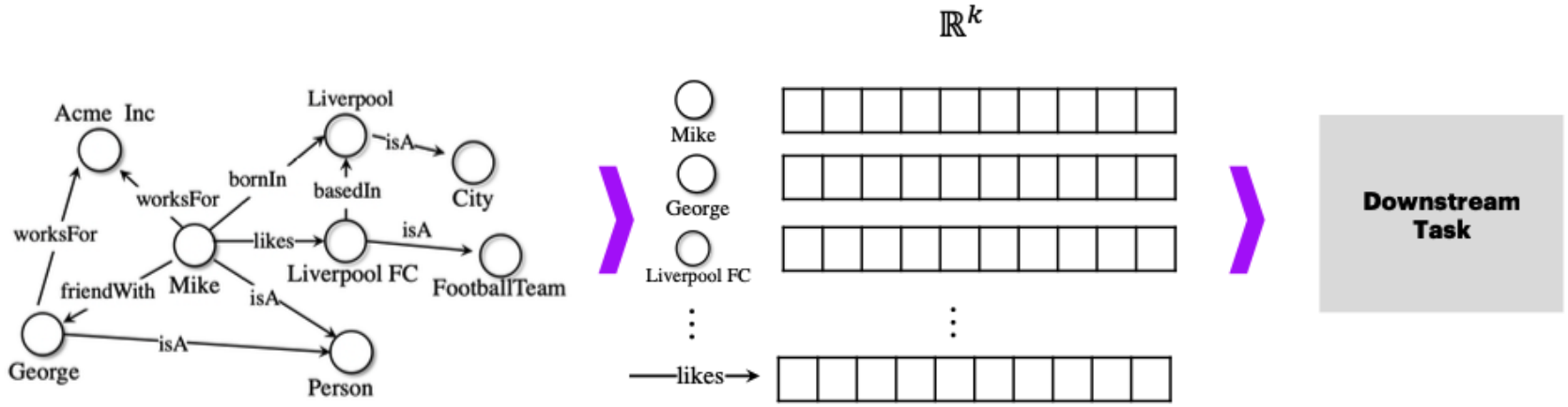
Limited Scalability to KG size
Limited modeling power
Non-differentiable approaches

ESILV

Graph (nodes and edges) representation learning

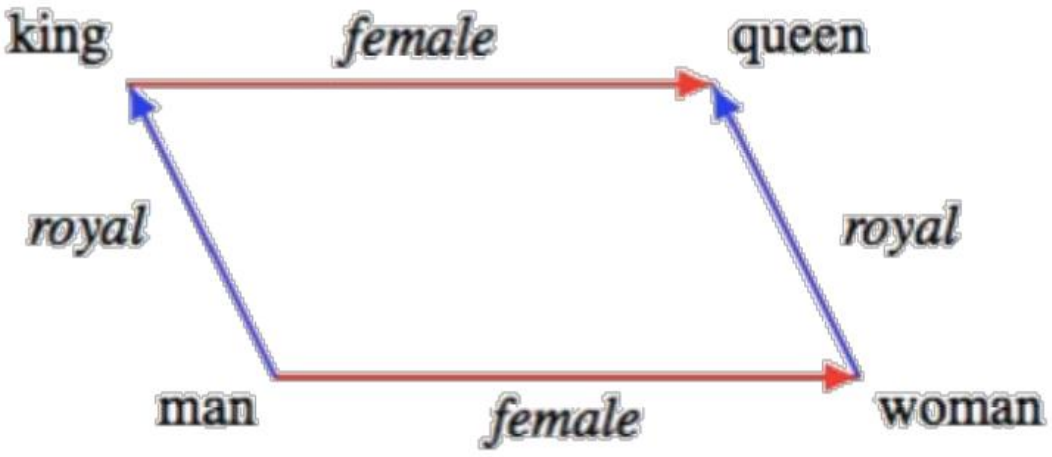
- Node Representation/Graph Feature based Methods
  - PRA, LINE, DeepWalk, node2vec
- Graph Neural Networks (GNNs)
  - GCNs, Graph Attention Networks
- **Knowledge Graph Embeddings (KGE)**
  - **TransE, DistMult, ComplEx, ConvE**

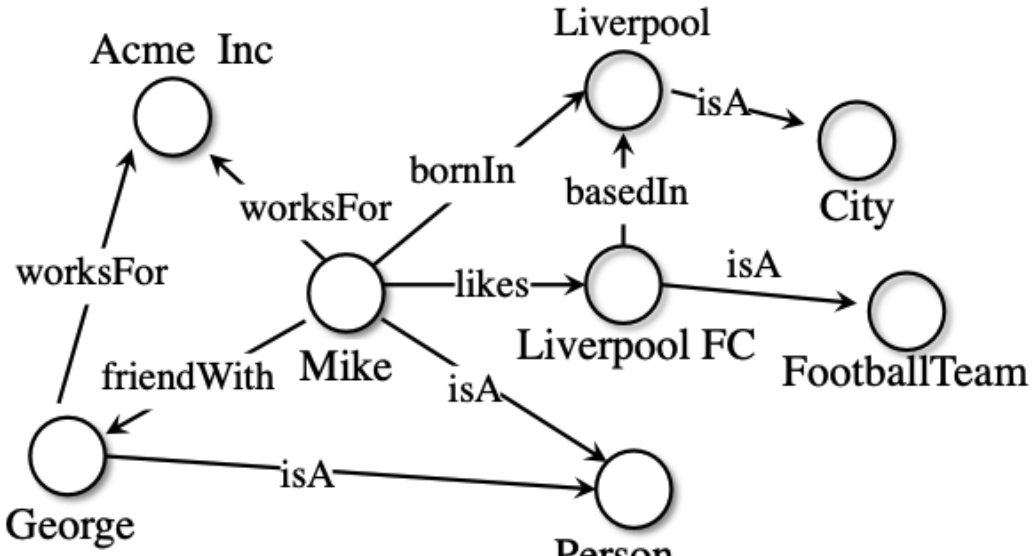
For a complete overview of graph feature-based models and GNNs: [Hamilton 2020]

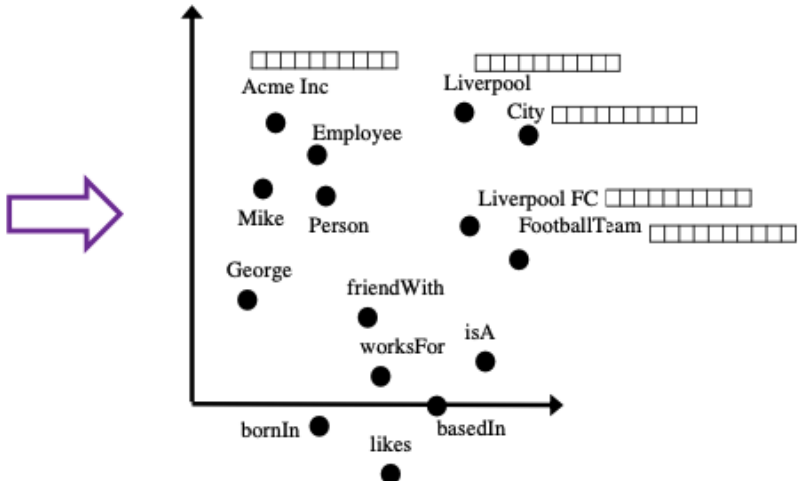


Another way: KG representation learning (embedding)

- If entities and relations are vectorized, (i.e. project entities and relations into a continuous low-dimensional space.) it would be possible to do the completion (link prediction).







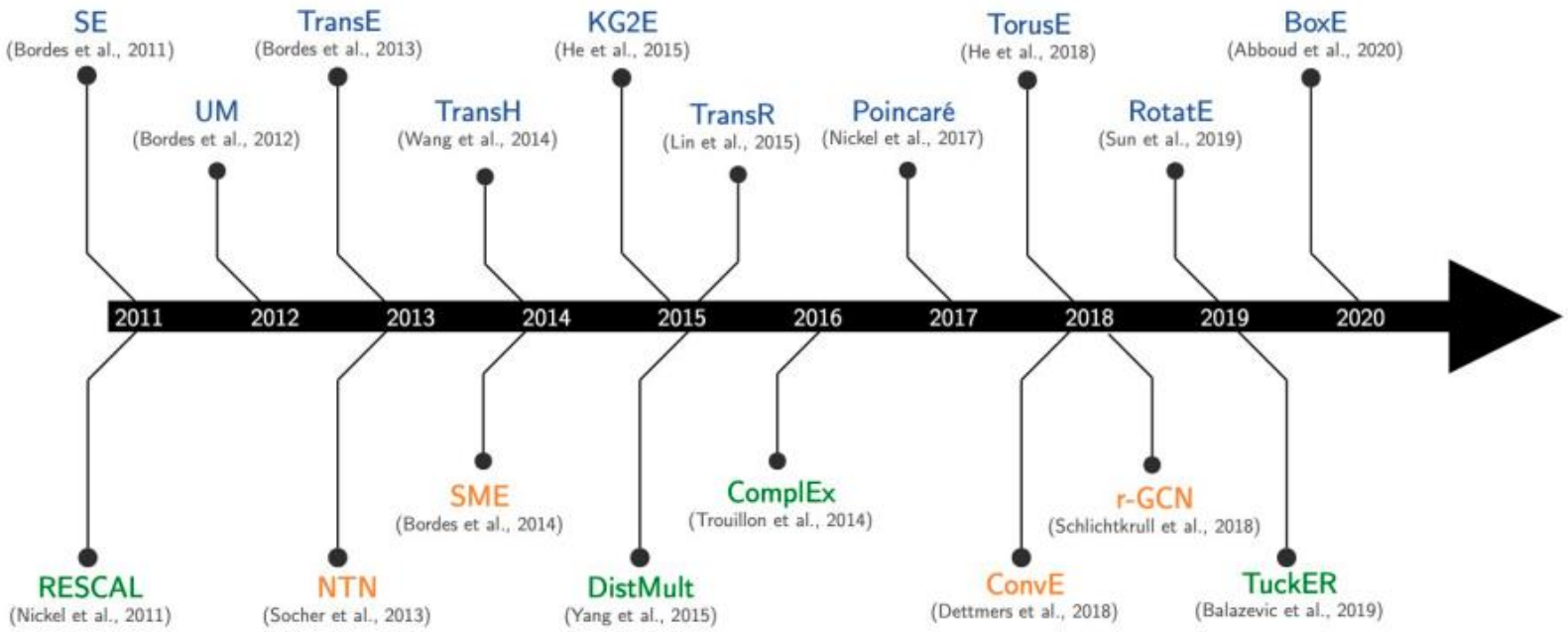
Recall: KG patterns (characteristics)

Symmetry	$r(s, o) \Rightarrow r(o, s)$	<Alice marriedTo Bob>
Antisymmetry	$r(s, o) \Rightarrow \neg r(o, s)$	<Alice childOf Jack>
Inversion	$r_2(s, o) \Rightarrow r_1(o, s)$	<Alice childOf Jack> <Jack fatherOf Alice>
Composition (transitivity)	$r_1(x,y) \wedge r_2(y,z) \Rightarrow r_3(x,z)$	<Alice childOf Jack> <Jack siblingOf Mary> <Alice nieceOf Mary>

- **But also:**
  - Hierarchies, Type constraints, Transitivity, Homophily, Long-range dependencies

ESILV

KG embedding models



Model	Score Function	Symmetry	Antisymmetry	Inversion	Composition
SE	$-\|\boldsymbol{W}_{r,1}\mathbf{h} - \boldsymbol{W}_{r,2}\mathbf{t}\|$	✗	✗	✗	✗
TransE	$-\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|$	✗	✓	✓	✓
TransX	$-\|g_{r,1}(\mathbf{h}) + \mathbf{r} - g_{r,2}(\mathbf{t})\|$	✓	✓	✗	✗
DistMult	$\langle \mathbf{h}, \mathbf{r}, \mathbf{t} \rangle$	✓	✗	✗	✗
ComplEx	$\mathrm{Re}(\langle \mathbf{h}, \mathbf{r}, \overline{\mathbf{t}} \rangle)$	✓	✓	✓	✗
RotatE	$-\|\mathbf{h} \circ \mathbf{r} - \mathbf{t}\|$	✓	✓	✓	✓

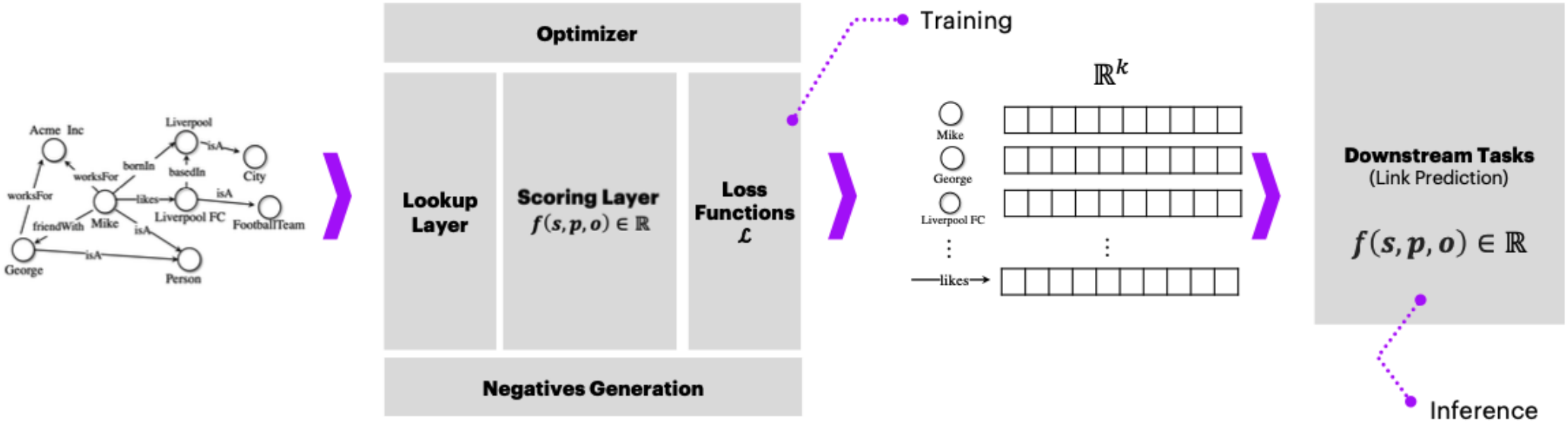
[Sun et al. 2019]

KG representation

- Edges in KG are represented as triples (*s, p, o*)
  - subject (s) predicate (p) object (o)
- Key Idea:
  - Model entities and relations in embedding space $\mathbb{R}^k$
    - Associate entities and relations with shallow embeddings
  - Given a triple (*s, p, o*), the goal is that the embedding of (s, p) should be close to the embedding of o.
    - How to embed s, p ?
    - How to define score *f(s,p)* ?
      - s.t. Score *f* is high if (s, p, o) exists, else *f* is low

ESILV

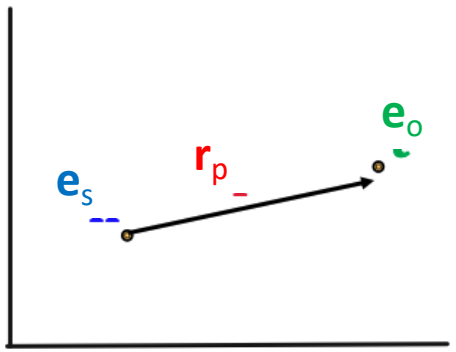
How KGE works

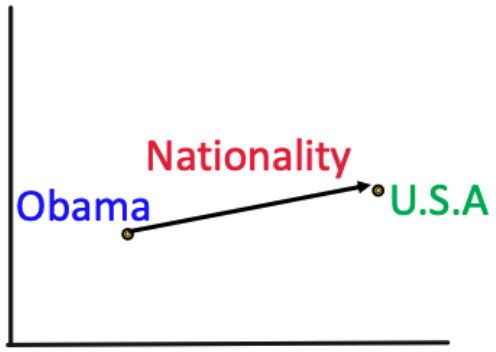


- Knowledge Graph (KG) $\mathcal{G}$
- Scoring function for a triple $f(t)$
- Loss function $\mathcal{L}$
- Optimization algorithm
- Negatives generation strategy

TRANSlating Embeddings (TransE)

- Intuition: Translation
- For a triple (s,p,o), let $\mathbf{e}_s, \mathbf{r}_p, \mathbf{e}_o \in \mathbb{R}^k$ be embedding vectors.
- TransE: $(\boldsymbol{e}_s + \boldsymbol{r}_p) \approx \boldsymbol{e}_o$ if the given link exists, else $(\boldsymbol{e}_s + \boldsymbol{r}_p) \neq \boldsymbol{e}_o$
  - Entity scoring function $f_{TransE} = -||(\boldsymbol{e}_s + \boldsymbol{r}_p) - \boldsymbol{e}_o||_n$





Algorithm

Algorithm 1 Learning TransE

input Training set $S =$ *{(s, p, o)}* , entities and rel. sets E and P, margin γ, embeddings dim. k.

initialize $p \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each $p \in P$

$p \leftarrow p/\|p\|$ for each $p \in P$

$e \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each entity $e \in E$

Initialize relations *p* and entities *e* uniformly, then normalize.
γ is the margin.

loop

$e \leftarrow e/\|e\|$ for each entity $e \in E$

$S_{batch} \leftarrow \text{sample}(S, b)$ // sample a minibatch of size b

$T_{batch} \leftarrow \emptyset$ // initialize the set of pairs of triplets

for *(s, p, o)* $\in S_{batch}$ **do**

(s′, p, o′) $\leftarrow \text{sample}(S'_{(s,p,o)})$ // sample a corrupted triplet

Sample triplet (s′, p, o′) that does not appear in the KG.

$T_{batch} \leftarrow T_{batch} \cup \{((s,p,o),(s',p,o'))\}$

end for

Update embeddings w.r.t.

$$\sum_{((s,p,o),(s',p,o')) \in T_{batch}} \nabla[\gamma + d(e_s + r_p, e_o) - d(e_{s'} + r_p, e_{o'})]_+$$

positive sample

negative sample

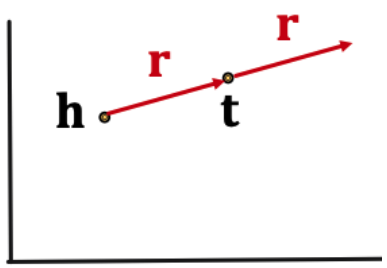
d represents distance (negative of score)

end loop

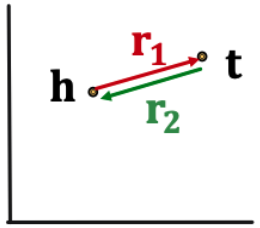
Contrastive loss: Favors lower distance (or higher score) for valid triplets, high distance (or lower score) for corrupted ones. We will see how to generate these corrupted triples later.

Are patterns also modelized?

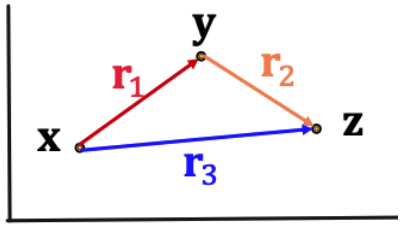
- Antisymmetric Relations: *r(s,o) ⇒ ¬r(o, s)* ✅
  - $e_s + r_p = e_o$, but $e_o + r_p \neq e_s$



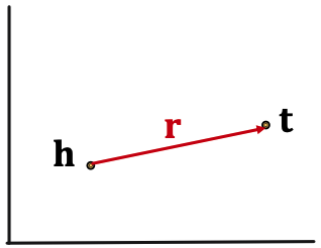
- Inverse Relations: $r_2(s,o) \Rightarrow r_1(o,s)$ ✅
  - $e_s + r_{p2} = e_o$ then we can set $r_{p2} = -r_{p1}$



- Composition (Transitivity) relation: $r_1(x,y) \wedge r_2(y,z) \Rightarrow r_3(x,z)$ ✅
  - $r_{p3} = r_{p1} + r_{p2}$



- Symmetric Relation: *r(s, o) ⇒ r(o, s)* ❌
  - TransE Cannot model Symmetric Relation
  - Only if $r_p = 0, e_s = e_o$



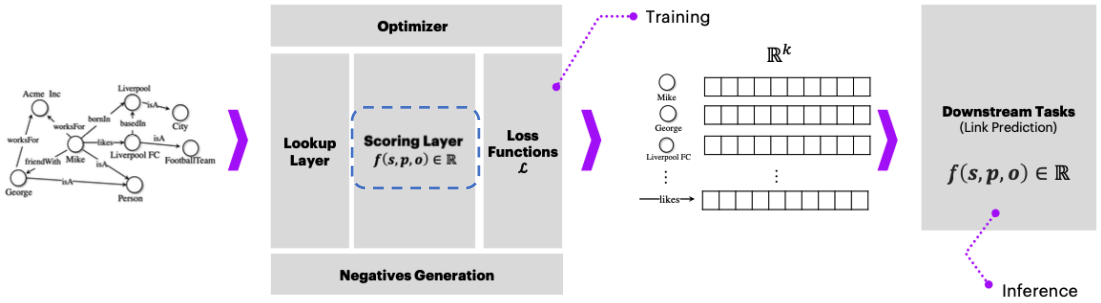
ESILV

A brief review on other embedding methods

- There are many KG embedding methods. What are their differences?
  - Score functions are different
  - Loss functions are different
  - Negatives generations methods are different

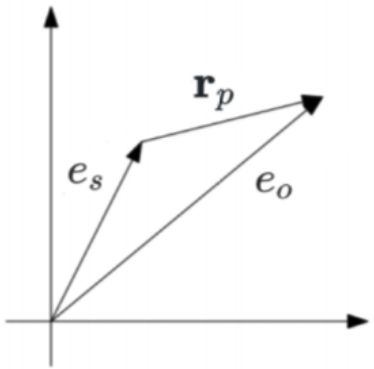
ESILV

Scoring function *f* (Translation-based)



- Recall: *f* assigns a score to a triple *(s, p, o)*
  - *f* is high if *(s, p, o)* exists, else *f* is low
- **TransE** [Bordes et al. 2013]

$$f_{TransE} = -||(\boldsymbol{e}_s + \boldsymbol{r}_p) - \boldsymbol{e}_o||_n$$



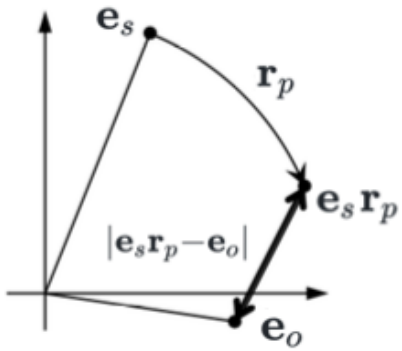
- **RotateE**: Relations modelled as rotations in complex space $\mathbb{C}$: element-wise product between complex embeddings. [Sun et al. 2019]

$$f_{RotateE} = -||\boldsymbol{e}_s \circ \boldsymbol{r}_p - \boldsymbol{e}_0||_n$$

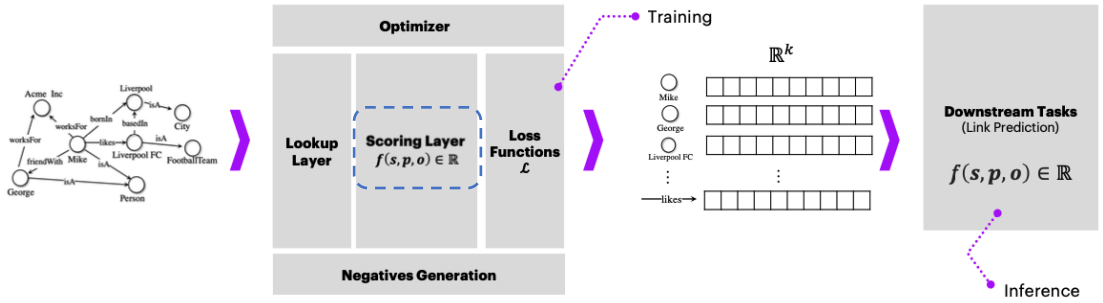
$\circ$ is the Hadmard (or element-wise) product

RotateE can modelize symmetry!
Proof: if r(x, y) and r(y, x) hold, we have

$$(\mathbf{y} = \mathbf{r} \circ \mathbf{x}) \wedge (\mathbf{x} = \mathbf{r} \circ \mathbf{y}) \Rightarrow \mathbf{r} \circ \mathbf{r} = 1$$

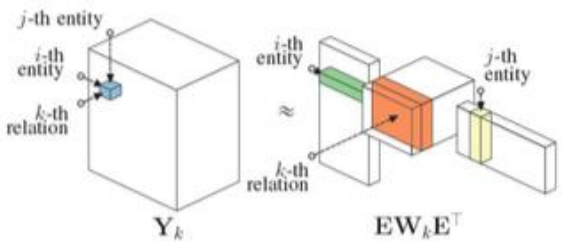


Scoring function f (Factorization-based)



- **RESCAL**: low-rank factorization with tensor product [Nickel et al. 2011]

$$f_{RESCAL} = \mathbf{e}_s^T \mathbf{W}_r \mathbf{e}_o$$



- **DistMult**: bilinear diagonal model. Dot product. [Yang et al. 2014]

$$f_{DistMult} = \langle \boldsymbol{r}_p, \boldsymbol{e}_s, \boldsymbol{e}_o \rangle$$

- **ComplEx**: Complex Embeddings (Hermitian dot product): (*i.e.* extends DistMult with dot product in C)

$$f_{ComplEx} = Re(\langle \boldsymbol{r}_p, \boldsymbol{e}_s, \boldsymbol{e}_o \rangle)$$

Deeper Scoring Functions

- **ConvE**: reshaping + convolution

[Dettmers et al. 2017]

Non-linearity

$$f_{ConvE} = \langle \sigma \left(vec \left(g \left([\overline{\mathbf{e}_s}; \overline{\mathbf{r}_p}] * \Omega \right) \right) \mathbf{W} \right) \right) \mathbf{e}_o \rangle$$

2D reshaping

Linear convolution

- **ConvKB**: convolutions and dot product

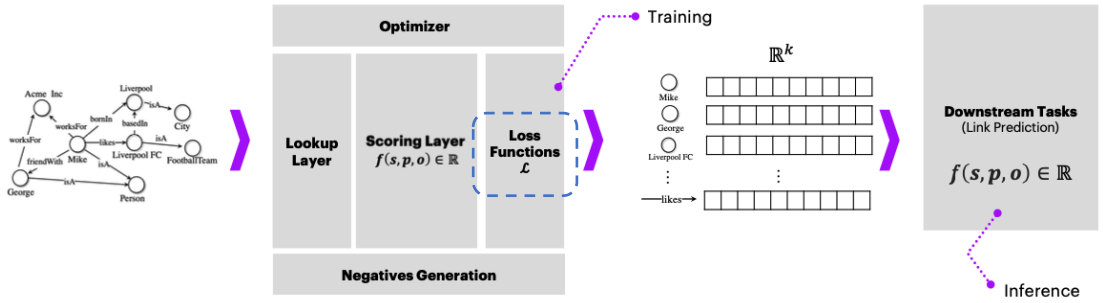
[Nguyen et al. 2018]

$$f_{ConvKB} = concat \left(g \left([\mathbf{e}_s, \mathbf{r}_p, \mathbf{e}_o] \right) * \Omega \right) \right) \cdot W$$

- Computationally expensive
- Others
  - HolE [Nickel et al. 2016]
  - SimplE [Kazemi et al. 2018]
  - QuatE [Zhang et al. 2019]
  - MurP [Balažević et al. 2019]

ESILV

Loss function



Pairwise Margin-Based Hinge Loss

Pays a penalty if score of positive triple < score of synthetic negative by a margin γ

$$\mathcal{L}(\Theta) = \sum_{t^+ \in \mathcal{G}} \sum_{t^- \in \mathcal{C}} max(0, [\gamma + \underbrace{f(t^-; \Theta)}_{\text{Score assigned to a synthetic negative}} - \underbrace{f(t^+; \Theta)}_{\text{Score assigned to true triple}}])$$

[Bordes et al. 2013]

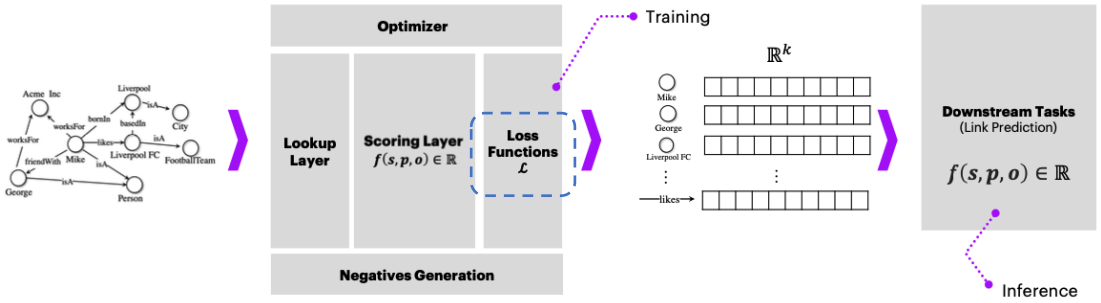
Negative Log-Likelihood / Cross Entropy

$$\mathcal{L}(\Theta) = \sum_{t \in \mathcal{G} \cup \mathcal{C}} log(1 + exp(-\underbrace{y}_{\text{Label of the triple t}} f(t; \Theta)))$$

$y \in \{-1, 1\}$

[Trouillon et al. 2016]

Loss function



Binary Cross-Entropy

$$\mathcal{L} = -\frac{1}{N} \sum_{t \in \mathcal{G} \cup \mathcal{C}}^{N} y \cdot log(\sigma(f(t;\Theta))) + (1-y) \cdot log(1 - f(t;\Theta))$$

[Dettmers et al. 2017]

Self-Adversarial

$$\mathcal{L} = -log\,\sigma(\gamma + f(t^+;\Theta)) - \sum_{t \in \mathcal{G}}^{N} \underbrace{p(t^-;\Theta)}_{\text{Weight for the negative sample } t^-} log\,\sigma(-f(t^-;\Theta) - \gamma)$$

[Sun et al. 2019]

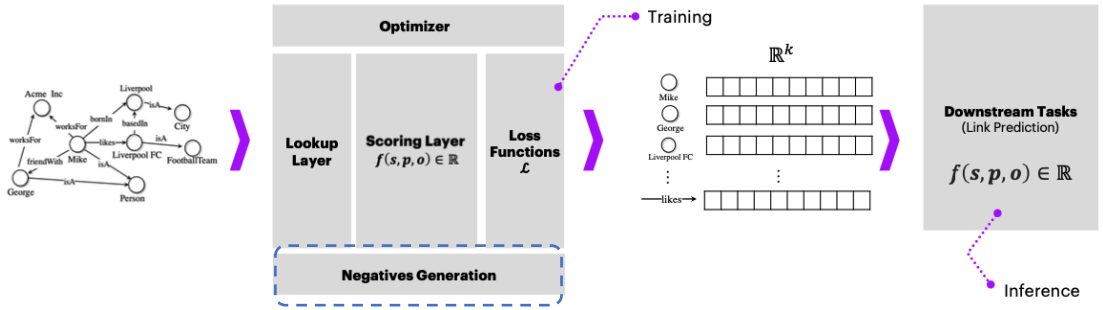
Many more: Multiclass Negative Log-likelihood, Absolute Margin, etc.

Regularizers and Initialization

- Regularizers
  - L1, L2
  - L3
  - Dropout (ConvE)
- Initialization
  - Random (Uniform)
  - Random (Normal)
  - Glorot

ESILV

Negatives Generation



- Where do negative examples come from? (i.e. false facts)
- "Corrupted" versions of a triple as synthetic negatives:

$$\mathcal{C} = \{(\hat{s}, p, o) | \hat{s} \in \mathcal{E}\} \cup \{(s, p, \hat{o}) | \hat{o} \in \mathcal{E}\}$$

"corrupted subject"

"corrupted" object

The predicate is unaltered

- Example

$$\mathcal{E} = \{Mike, Liverpool, AcmeInc, George, LiverpoolFC\}$$
$$\mathcal{R} = \{bornIn, friendWith\}$$

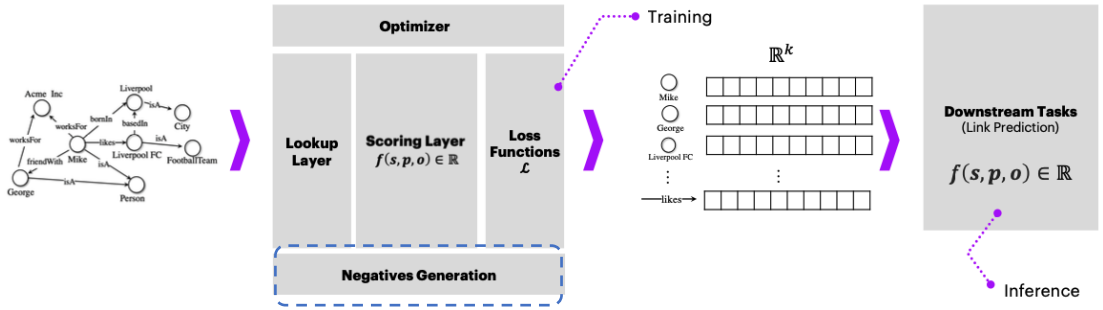
$t \in \mathcal{G} =$ `(Mike bornIn Liverpool)`

$\mathcal{C}_t =$

```
Mike      bornIn   AcmeInc
Mike      bornIn   LiverpoolFC
George    bornIn   Liverpool
AcmeInc   bornIn   Liverpool
```

✓/✗ → Margin

Training procedure



- **Training with synthetic negaives:**
  - **Uniform sampling**: generate all possible synthetic negatives and sample n negatives for each positive t.
  - **Complete set**: no sampling. Use all possible synthetic negatives for each positive t. (This may bring scalability problems)
  - **1-n scoring**: batches of (s, p, *) or (*, p, o) labeled as positives (if included in training KG) or negatives (if not in training KG). [Dettmers et al. 2018]
- Optimizer
  - learn optimal parameters (e.g. embeddings). SGD variants: (AdaGrad, Adam)

$$\min_{\Theta} \mathcal{L}(\Theta)$$

Evaluation

- **Learning–To-Rank problem:**
  How well are positive triples ranked against **synthetic negatives** built under the **Local Closed World Assumption.**
- **Evaluation Metrics**
  - **Mean Rank (MR)**

$$MR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} rank_{(s,p,o)_i}$$

  - **Mean Reciprocal Rank (MRR)**

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_{(s,p,o)_i}}$$

  - **Hits@N**

$$Hits@N = \sum_{i=1}^{|Q|} 1 \text{ if } rank_{(s,p,o)_i} \leq N$$

ESILV

Evaluation: Example

- How unseen, test positive triples rank against **synthetic negatives**?

s	p	o	score	rank	
Mike	born_in	Leeds	0.789	1	
Mike	born_in	Liverpool	0.753	2	*
Mike	born_in	Germany	0.695	3	
George	born_in	Liverpool	0.456	4	
Mike	born_in	George	0.234	5	

s	p	o	score	rank	
Mike	friend_with	George	0.901	1	*
Mike	friend_with	Jim	0.345	2	
Acme	friend_with	George	0.293	3	
Mike	friend_with	Liverpool	0.201	4	
France	friend_with	George	0.156	5	

Unseen positive triples (test set)

$$MR = 1.5$$
$$MRR = 0.75$$
$$Hits@1 = 0.5$$
$$Hits@3 = 1.0$$

ESILV

Benchmark Datasets and SOTA results

Freebase WordNet YAGO

	FB15K-237	WN18RR	YAGO3-10
Training	272,115	86,835	1,079,040
Validation	17,535	3,034	5,000
Test	20,466	3,134	5,000
Entities	14,541	40,943	123,182
Relations	237	11	37

	FB15K-237	WN18RR	YAGO3-10
Literature Best	**0.35***	0.48*	0.49*
TransE (AmpliGraph)	0.31	0.22	**0.51**
DistMult (AmpliGraph)	0.31	0.47	0.50
ComplEx (AmpliGraph)	0.32	**0.51**	0.49
HolE (AmpliGraph)	0.31	0.47	0.50
ConvE (AmpliGraph)	0.26	0.45	0.30
ConvE (1-N, AmpliGraph)	0.32	0.48	0.40
ConvKB (AmpliGraph)	0.23	0.39	0.30

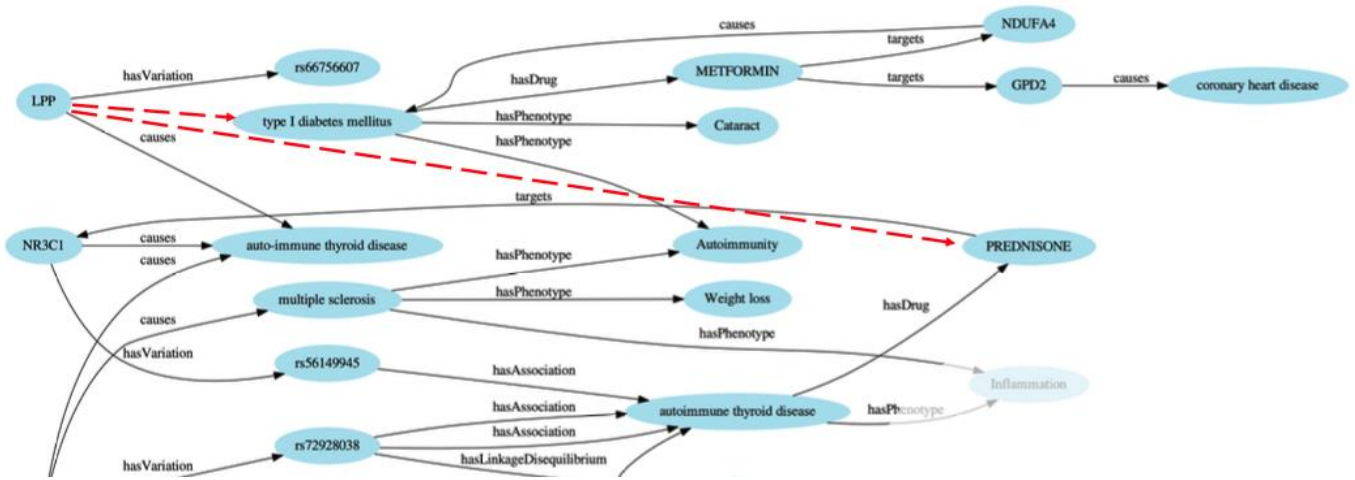
[https://github.com/Accenture/AmpliGraph]

		FB15K-237		*WNRR*	
		MRR	Hits@10	MRR	Hits@10
First	RESCAL (Wang et al., 2019)	27.0	42.7	42.0	44.7
	TransE (Nguyen et al., 2018)	29.4	46.5	22.6	50.1
	DistMult (Dettmers et al., 2018)	24.1	41.9	43.0	49.0
	ComplEx (Dettmers et al., 2018)	24.7	42.8	44.0	51.0
	ConvE (Dettmers et al., 2018)	32.5	50.1	43.0	52.0
Ours	RESCAL	**35.7**	**54.1**	46.7	51.7
	TransE	31.3	49.7	22.8	52.0
	DistMult	34.3	53.1	45.2	53.1
	ComplEx	34.8	53.6	**47.5**	**54.7**
	ConvE	33.9	52.1	44.2	50.4
Recent	TuckER (Balazevic et al., 2019)	**35.8**	**54.4**	47.0	52.6
	RotatE (Sun et al., 2019a)	33.8	53.3	**47.6**	**57.1**
	SACN (Shang et al., 2019)	35.0	54.0	47.0	54.4
Large	DistMult (Salehi et al., 2018)	35.7	54.8	45.5	54.4
	ComplEx-N3 (Lacroix et al., 2018)	**37.0**	**56.0**	**49.0**	**58.0**

[Ruffinelli et a. 2020]

Application

- Pharmaceutical Industry:
  - Drug Discovery
  - Drug Side Effect Prediction
  - Determining risk factors
- Product recommendation
- Social network community
- Inferences/recommendation/prediction…any completion of graphs benefit from this technique.



ESILV

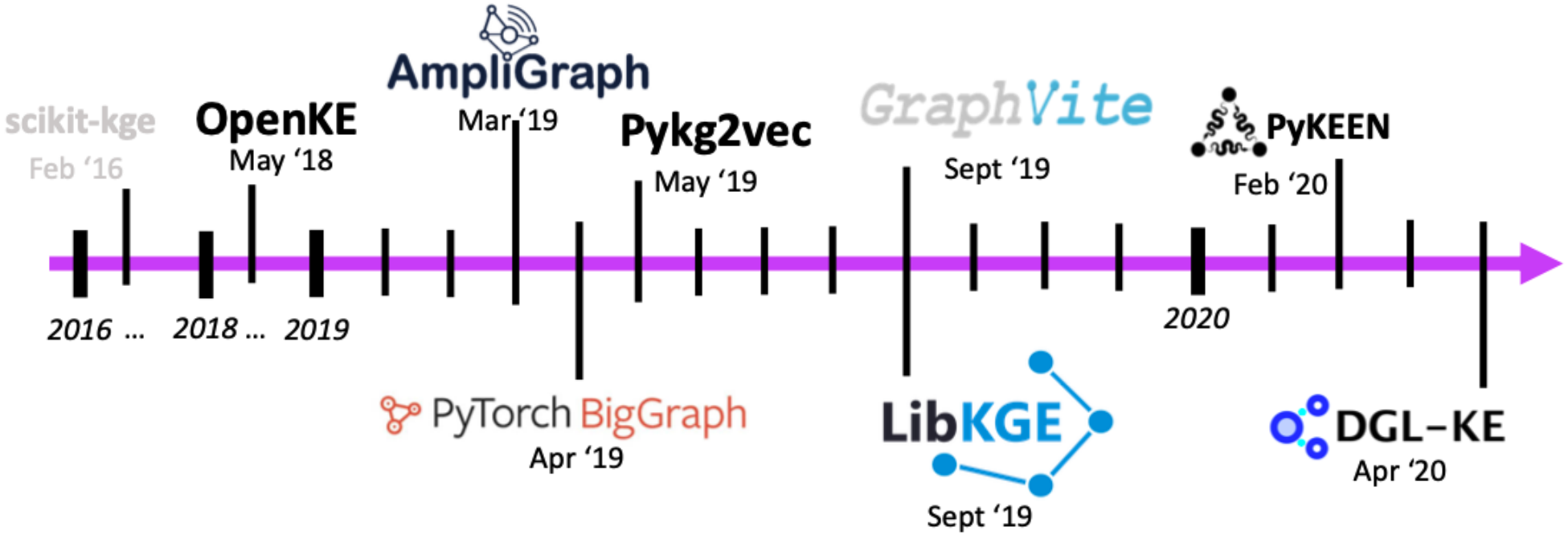
Inference tasks

- What embeddings infer:
  - Missing links (link prediction)
  - Missing types (entity classification) (prediction of link ”typeOf”)
  - Entity similarity (any similarity between vectors)
  - Relation prediction (prediction of links related to relations)
- Contrast with SWRL
  - SWRL infers **deterministic** new triples
  - Embeddings infer **probable** new triples
- Comparison on the right ->

Aspect	SWRL / Symbolic	Embeddings
Precision	Exact, deterministic	Approximate
Explainability	High	Low
Scalability	Limited	Excellent
Handling noise	Poor	Good
Prerequisites	Expert-written rules	Training data
Output	Guaranteed truths	Probabilistic predictions

Implementation and practice

- KGE's software universe



Models

	TransE	DistMult	ComplEx	TransH	TransD	TransR	RESCAL	HolE	SimplE	Analogy	ConvKB	ConvE
OpenKE	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗
AmpliGraph	✓	✓	✓	✗	✗	✗	✗	✓	✗	✗	✓	✓
PyTorch BigGraph	✓	✓	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗
GraphVite	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗
DGL-KE	✓	✓	✓	✗	✗	✓	✓	✗	✗	✗	✗	✗
PyKEEN	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
Pykg2vec	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Lib-KGE	✓	✓	✓	✗	✗	✗	✓	✗	✓	✗	✗	✓
scikit-kge	✓	✗	✗	✗	✗	✓	✓	✓	✗	✗	✗	✗

ESILV

Models

	RotatE	QuatE	KG2E	NTN	ProjE	RGCN	TuckER	TransM	CP	Other models
OpenKE	✗	✗	✗	✗	✗	✗	✗	✗	✗	
AmpliGraph	✗	✗	✗	✗	✗	✗	✗	✗	✗	
PyTorch BigGraph	✗	✗	✗	✗	✗	✗	✗	✗	✗	
GraphVite	✓	✓	✗	✗	✗	✗	✗	✗	✗	
DGL-KE	✓	✗	✗	✗	✗	✗	✗	✗	✗	
PyKEEN	✓	✗	✓	✓	✓	✓	✓	✗	✗	Complex /DistMultLiteral, ERMLP, StructuredEmbedding, SME
Pykg2vec	✓	✗	✓	✓	✓	✗	✓	✓	✓	SLM, SME, ComplexN3
Lib-KGE	✓	✗	✗	✗	✗	✗	✓	✗	✓	
scikit-kge	✗	✗	✗	✗	✗	✗	✗	✗	✗	ERMLP

ESILV

Pre-trained Models

	WikiData dump	Freebase	Benchmark datasets
OpenKE	✓ link	✓ (fragment ?)	✗
AmpliGraph	✗	✗	✓ (upon request)
PyTorch BigGraph	✓ (full)	✗	✗
GraphVite	✓ (Wikidata5m)	✗	✗
DGL-KE	✗	✗	✗
PyKEEN	✗	✗	✗
Pykg2vec	✗	✗	✗
Lib-KGE	✓ (Wikidata5M)	✗	✓ link
scikit-kge	✗	✗	✗

ESILV

Scalability

	Core Framework	GPU	Distributed Execution CPU	Biggest Graph
OpenKE	PyTorch/Tensorflow	✓	✗	10^8 edges, $4x10^7$ nodes [3]
AmpliGraph	Tensorflow	✓	✗ (Coming)	10^8 edges, 10^6 nodes [1]
PyTorch BigGraph	PyTorch	✓	✓	$2.4x10^{12}$ edges, $1.21x10^8$ nodes [4]
GraphVite	PyTorch	✓	✓ (GPU-CPU)	$1.8x10^{12}$ edges, $6.6x10^6$ nodes [9]
DGL-KE	PyTorch	✓	✓	$3.38x10^8$ edges, $8.6x10^6$ nodes [8]
PyKEEN	PyTorch	✓	✗	-
Pykg2vec	PyTorch/Tensorflow	✓	✗	-
Lib-KGE	PyTorch	✓	✗	-
scikit-kge	-	✗	✗	-

ESILV

Rules + Learning: hybrid approaches

- Hybrid reasoning integrates **symbolic** (logic-based) inference with **statistical** (embedding/neural) inference to create systems that are:
  - **explainable** (symbolic rules, constraints)
  - **robust to noise** (neural/embedding learning)
  - **generalizing** (statistical models)
  - **correct-by-design** (logical constraints)

- We give only brief introduction in this lecture

ESILV

Symbolic → Guides → Embeddings

- Symbolic rules constrain or shape embedding training to ensure learned embeddings **respect logical knowledge**.
- Use **rule regularization** by adding rule-based constraints to the loss. L_total = L_embedding + λ * L_rule
  - E.g. a horn rule: r1(x,y) ∧r2(y,z) → r3(x,z).
  - Its embedding-based translation is: score_r1(x,y) * score_r2(y,z) ≤ score_r3(x,z)
  - The penalty term is added: L_rule = max(0, score_r1(x,y)*score_r2(y,z) - score_r3(x,z))
- Use **semantic regularizers** to impose properties/characteristics
  - E.g. for relations with symmetric property (such as marriedTo) you can define score(h, r, t) ≈ score(t, r, h)
- Some representative methods:
  - **ComplEx-NNE+AER** (embeddings + axiomatic constraints)
  - **RUGE** (iterative rule-guided embedding learning)
  - **IterE** (learning rules ↔ embeddings iteratively)

ESILV

Embeddings → Discover → Symbolic Rules

- Embeddings help discover new rules, such as property chains, type constraints, Horn rules, inverse relations, symmetric/asymmetric relations
- if for many triples: the relation $h + r1 + r2 \approx t$ applies, then suggest $r1(x,y) \wedge r2(y,z) \rightarrow r3(x,z)$
- Benefits:
  - Automates rule discovery (reduces expert workload)
  - Extracts emergent patterns from data
- Representative methods:
  - **AMIE+** (rule mining; not neural but widely used)
  - **Neural LP** (learn logical rules with differentiable operators)
  - **DRUM** (differentiable rule mining through RNNs)
  - **RNNLogic**

ESILV

References (publications)

- DeLong, L. N., Mir, R. F., & Fleuriot, J. D. *Neurosymbolic AI for Reasoning over Knowledge Graphs: A Survey*. arXiv (2023)
- Shu Guo, Quan Wang, Lihong Wang, Bin Wang, Li Guo. *Knowledge Graph Embedding with Iterative Guidance from Soft Rules.* arXiv (2017)
- Luis Galárraga, Christina Teflioudi, Katja Hose, Fabian Suchanek. *Fast Rule Mining in Ontological Knowledge Bases with AMIE+.* The VLDB Journal 24, no. 6 (2015): 707-730.
- Wen Zhang, Bibek Paudel, Liang Wang, Jiaoyan Chen, Hai Zhu, Wei Zhang, Abraham Bernstein, Huajun Chen. *Iteratively Learning Embeddings and Rules for Knowledge Graph Reasoning.* WWW 2019
- Fan Yang, Zhilin Yang, William W. Cohen. *Differentiable Learning of Logical Rules for Knowledge Base Reasoning*. NIPS 2017.
- Ali Sadeghian, Mohammadreza Armandpour, Patrick Ding, Daisy Zhe Wang. *DRUM: End-To-End Differentiable Rule Mining On Knowledge Graphs.* NIPS 2019
- Meng Qu, Junkun Chen, Louis-Pascal Xhonneux, Yoshua Bengio, Jian Tang. *RNNLogic: Learning Logic Rules for Reasoning on Knowledge Graphs.* ICLR2021

ESILV

References (courses)

- Daniel Jurafsky & James H. Martin. Ch.6, Speech and Language Processing.
- Julia Hockenmaier, CS546 Lecture 4: static word embeddings, Illinois Univ.
- Luke Zettlemoyer, CSEP517 Word Embeddings, Washington Univ.
- Charilaos Kanatsoulis and Jure Leskovec,CS224W, Knowledge Graph Embeddings, Stanford University
- Luca Costabello, Knowledge Graph Embeddings: From Theory to Practice, ECAI2020

ESILV